



Constraints

Constraints let you define the way the Database Engine **automatically enforces the integrity** of a database.

- **Constraints define rules** regarding the values allowed in columns and are the standard mechanism for enforcing integrity.
- Using constraints is preferred to using DML Triggers, rules, and defaults.
- The query optimizer also uses constraint definitions to build high-performance query execution plans.
- **Constraints are tested before** new row is added or an existing row updated or deleted.

Constraints – Table vs Column

Constraints can be **column constraints** or **table constraints**.

- A **column constraint** is specified as **part of a single column definition** and applies only to that column.
- A **table constraint** is **declared independently** from a column definition and **can apply to more than one column in a table**.
- **Table-level constraints** must be used when more than one column must be included in a constraint.

Column Values with Constraints

We've discussed **primary keys** and **foreign keys** to establish **declarative relationships** between tables.

Other constraints include:

- **CHECK**: Allows only those rows where a supplied Boolean expression evaluates to true
- **UNIQUE**: Ensures that values in a column or group of columns are unique in each row in the table. This does allow a NULL value in the column.
- **NOT NULL**: Prevents NULL values in a column.

Using a UUID as a Surrogate Key

PostgreSQL offers a **UUID** (Universally Unique Identifier) data type as well as two modules that generate UUIDs: `uuid-osp` and `pgcrypto`.

```
2911d8a8-6dea-4a46-af23-d64175a08237
```

Exercise caution when considering UUIDs for a surrogate key. Because of their size, they are inefficient compared with options such as `int` or `bigint`.

A Single-Column Primary Key

```
CREATE TABLE natural_key_example (  
    license_id text CONSTRAINT license_key PRIMARY KEY  
    , first_name text  
    , last_name text  
);
```

Column constraint

The constraint is entered with the column definition.

```
CREATE TABLE natural_key_example (  
    license_id text  
    , first_name text  
    , last_name text  
    , CONSTRAINT license_key PRIMARY KEY (license_id)  
);
```

Table constraint

The constraint is entered below the column definition.

Without Naming the Constraint

```
CREATE TABLE natural_key_example (  
    license_id text PRIMARY KEY  
    , first_name text  
    , last_name text  
);
```

Column constraint

The constraint is entered with the column definition.

```
CREATE TABLE natural_key_example (  
    license_id text  
    , first_name text  
    , last_name text  
    , PRIMARY KEY (license_id)  
);
```

Table constraint

The constraint is entered below the column definition.

A Composite Primary Key

```
CREATE TABLE natural_key_composite_example (  
    student_id text  
    , school_day date  
    , present boolean  
    , CONSTRAINT student_pk  
    PRIMARY KEY (student_id, school_day)  
);
```

For **composite primary keys**, we can only create table constraints.

Auto-Incrementing Surrogate Keys

There are two ways to have a PostgreSQL database add an automatically increasing unique value to a column.

1. Set the column to one of the PostgreSQL specific serial data types: `smallserial`, `serial`, and `bigserial`. Not part of the SQL Standard. **On the exam, I won't accept this form.**
2. **Use the IDENTITY syntax.** This is part of the SQL Standard.

```
order_number bigint GENERATED ALWAYS AS IDENTITY
```

Auto-Incrementing Surrogate Keys

```
CREATE TABLE surrogate_key_example (  
    order_number bigint  
        GENERATED ALWAYS AS IDENTITY primary key  
    , product_name text  
    , order_time timestamp with time zone  
);
```

Column constraint

```
CREATE TABLE surrogate_key_example (  
    order_number bigint  
        GENERATED ALWAYS AS IDENTITY  
    , product_name text  
    , order_time timestamp with time zone  
    , CONSTRAINT order_number_key PRIMARY KEY (order_number)  
);
```

Table constraint

About Those Auto-Incrementing Keys

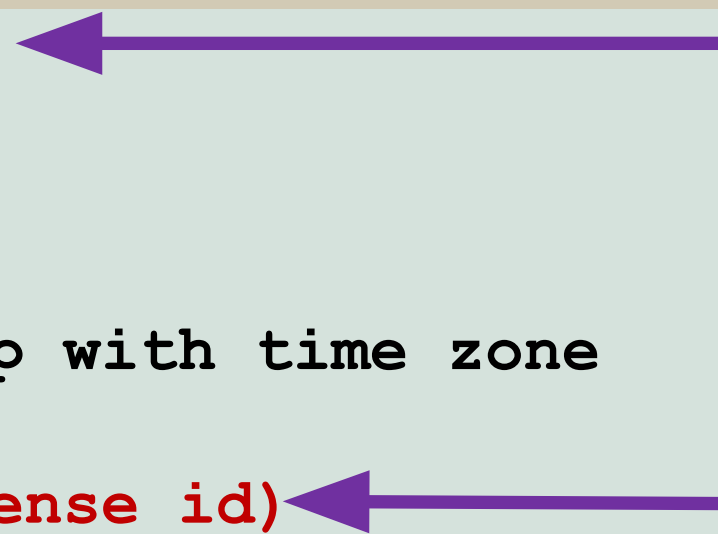
- The auto-increment value is monotonically increasing.
- It will generally add one to the largest existing value in the sequence.
- If you delete a row, the **database won't fill the gap in the `order_number` sequence, nor will it change any of the existing values in that column.**
- Since a surrogate key, by its very nature, has no intrinsic meaning, having gaps in the numbers is fine.

Foreign Keys

- We use **foreign keys to establish relationships** between tables.
- A **foreign key is one or more columns whose values match those in another table's primary key or other unique key.**
- **Foreign key values must already exist in the primary key or other unique key of the table it references.**
 - If not, the value is rejected.
- With this constraint, **SQL enforces referential integrity** — ensuring that data in related tables does not end up unrelated, or orphaned.

Foreign Keys

```
CREATE TABLE licenses (  
    license_id text  
    , first_name text  
    , last_name text  
    , CONSTRAINT licenses_key  
      PRIMARY KEY (license_id)  
);  
  
CREATE TABLE registrations (  
    registration_id text  
    , registration_date timestamp with time zone  
    , license_id text  
      REFERENCES licenses (license_id)  
    , CONSTRAINT registration_key  
      PRIMARY KEY (registration_id, license_id)  
);
```



This is called **declarative referential integrity**. We explicitly declared it as part of the structure of the tables, how they are related and have the database enforce the rules.

Cascading Deletes

```
CREATE TABLE registrations (  
    registration_id text  
    , registration_date date  
    , license_id text  
        REFERENCES licenses (license_id) ON DELETE CASCADE  
    , CONSTRAINT registration_key  
        PRIMARY KEY (registration_id, license_id)  
);
```



Deleting a row in **licenses** should also delete all related rows in **registrations**. This allows us to delete a driver's license without first having to remove any registrations linked to it. It also maintains data integrity by ensuring deleting a license **doesn't leave orphaned rows in registrations**.

The **ON DELETE** clause in PostgreSQL supports the following actions:

- **SET NULL**: automatically sets **NULL** to the foreign key columns in the referencing rows of the child table when the referenced rows in the parent table are deleted.
- **SET DEFAULT**: sets the **default value** to the foreign key column of the referencing rows in the child table when the referenced rows from the parent table are deleted.
- **NO ACTION**: issues a **constraint violation** when deleting rows from the **pk** table when there are rows in the **fk** table.
- **RESTRICT**: The RESTRICT action is similar to the NO ACTION. The difference is when you define the foreign key constraint as DEFERRABLE with an INITIALLY DEFERRED or INITIALLY IMMEDIATE mode.
- **CASCADE**: deletes data from a child table automatically when the parent data are deleted.

CHECK Constraints

A **CHECK** constraint evaluates whether data added to a column meets the expected criteria, which we specify with a logical test. If the criteria are not met, the database returns an error.

```
CREATE TABLE check_constraint_example (  
    user_id bigint GENERATED ALWAYS AS IDENTITY  
    , user_role text  
    , salary numeric(10,2)  
    , CONSTRAINT user_id_key PRIMARY KEY (user_id)  
    , CONSTRAINT check_role_in_list  
        CHECK (user_role IN('Admin', 'Staff'))  
    , CONSTRAINT check_salary_not_below_zero  
        CHECK (salary >= 0)  
);
```



UNIQUE Constraints

- We can also ensure that a column has a unique value in each row by using the **UNIQUE** constraint.
- If ensuring **unique** values sounds *similar* to the purpose of a primary key, it is.
- **UNIQUE** has an important difference.
 - In a primary key, no values can be NULL, but **a UNIQUE constraint permits multiple NULL values in a column**

```
CREATE TABLE unique_constraint_example (  
    contact_id bigint GENERATED ALWAYS AS IDENTITY  
    , first_name text  
    , last_name text  
    , email text  
    , CONSTRAINT contact_id_key PRIMARY KEY (contact_id)  
    , CONSTRAINT email_unique UNIQUE (email)  
);
```

The NOT NULL Constraint

To require a value in a column not allow the NULL value, SQL provides the **NOT NULL** constraint, which simply prevents a column from accepting empty values.

```
CREATE TABLE not_null_example (  
    student_id bigint GENERATED ALWAYS AS IDENTITY  
    , first_name text NOT NULL  
    , last_name text NOT NULL  
    , CONSTRAINT student_id_key  
      PRIMARY KEY (student_id)  
    , CONSTRAINT name_unique  
      UNIQUE (last_name, first_name)  
);
```

Adding Constraints Outside the Table Definition

```
ALTER TABLE not_null_example  
    DROP CONSTRAINT student_id_key;
```

```
ALTER TABLE not_null_example  
    ADD CONSTRAINT student_id_key PRIMARY KEY  
    (student_id);
```

```
ALTER TABLE not_null_example  
    ALTER COLUMN first_name DROP NOT NULL;
```

```
ALTER TABLE not_null_example  
    ALTER COLUMN first_name SET NOT NULL;
```

Generated Columns

CREATE TABLE supports the specification of **generated** columns.

Values of a **generated** column are **computed from an expression** included in the column definition.

```
create table triangle (  
    sidea float  
    , sideb float  
    , sidec float GENERATED ALWAYS AS  
        (SQRT(sidea * sidea + sideb * sideb)) STORED  
);  
insert into triangle (sidea, sideb)  
values (1,1) , (3,4) , (6,8) ;
```

Generated Columns

```
insert into triangle (sidea, sideb)
values (1,1) , (3,4) , (6,8) ;
```

```
select *
from chapter8_examples.triangle
```

sidea	sideb	sidec
1	1	1.4142135624
3	4	5
6	8	10

Indexes on Tables

- Each time you **add a primary key or unique constraint**, PostgreSQL (as well as most database systems) **creates an index on the column or columns included in the constraint**.
- **Indexes are stored separately from the table data** and are accessed automatically (if needed) when you run a query and updated every time a row is added, removed, or updated.
- In PostgreSQL, the default index type is the B-tree index.

Considerations When Using Indexes

- PostgreSQL, for example, has five more index types in addition to B-tree.
- Primary keys and Unique keys are indexed by default in PostgreSQL, **but foreign key columns in related tables are not** and are a good subject for indexes.
- **Add indexes to columns that will frequently end up in a query WHERE clause.** Search performance can be significantly improved using indexes.
- Having **too many indexes on a table will not only take up more space, but can slow operations**, such as inserts and deletes.
 - Adding a new row will require indexes that use that row be updated.
 - Deleting a row from a table requires indexes be updated as well.
 - Maintenance on indexes must be done to ensure they continue to aid in searches on the table.